



Naive Bayes

Lecture-4
Machine Learning (H9MLAI)

Naïve Bayes Algorithm

Introduction

Probability

Naïve Bayes

Examples

Summary

Introduction

▶ Background

- ▶ When a meteorologist in the US provides a weather forecast, precipitation is typically predicted using terms such as "70 percent chance of rain."
- ▶ These estimates are based on probabilistic methods, or methods concerned with describing uncertainty. They use data on past events to extrapolate future events.
- ▶ In the case of weather, the chance of rain describes the proportion of prior days with similar measurable atmospheric conditions in which precipitation occurred.
- ▶ A 70 percent chance of rain therefore implies that in 7 out of 10 past cases with similar weather patterns, precipitation occurred somewhere in the area.



Introduction

▶ Naïve Bayes

- ▶ Naive Bayes uses data about prior events to estimate the probability of future events.
 - For instance, a common application of naive Bayes uses the frequency of words in past junk email messages to identify new junk mail.
- ▶ What will we cover:
 - Basic principles of probability that are utilized for naive Bayes
 - Specialized methods, visualizations, and data structures used for analyzing text data with python
 - How to employ an python implementation of naive Bayes classifier to build an message filter



Probability

▶ Bayes & Probability

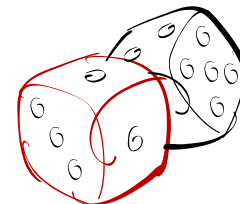
- ▶ Classifiers based on Bayesian methods utilize training data to calculate an observed probability of each class based on feature values.
- ▶ When the classifier is used later on unlabeled data, it uses the observed probabilities to predict the most likely class for the new features.
- ▶ It's a simple idea, but it results in a method that often has results on par with more sophisticated algorithms.
- ▶ In fact, Bayesian classifiers have been used for:
 - Text classification, such as junk email (spam) filtering, author identification, or topic categorization
 - Intrusion detection or anomaly detection in computer networks
 - Diagnosing medical conditions, when given a set of observed symptoms

Introduction

▶ Bayes & Probability

- ▶ Typically, Bayesian classifiers are best applied to problems in which the information from numerous attributes should be considered simultaneously in order to estimate the probability of an outcome.
- ▶ While many algorithms ignore features that have weak effects, Bayesian methods utilize all available evidence to subtly change the predictions.
- ▶ If a large number of features have relatively minor effects, taken together their combined impact could be quite large.
- ▶ Bayesian probability theory is rooted in the idea that the estimated likelihood of an event should be based on the evidence at hand.
 - **Events** are possible outcomes, such as sunny and rainy weather, a heads or tails result in a coin flip, or spam and not spam email messages.
 - A **trial** is a single opportunity for the event to occur, such as a day's weather, a coin flip, or an email message.

Probability



► Probability Recap

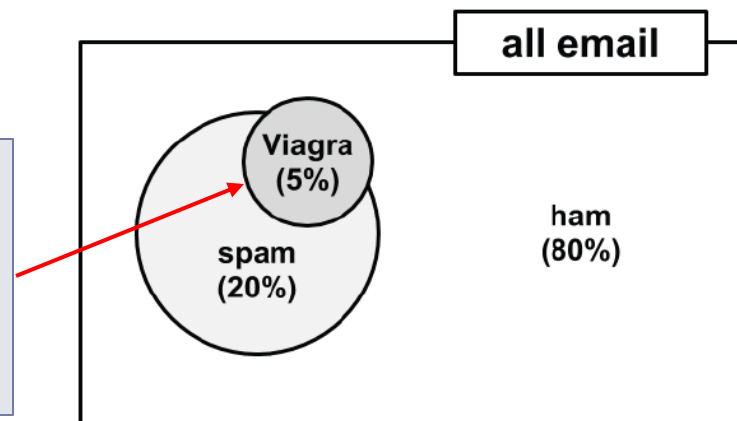
- A quick recap! The *probability* of an event can be estimated from observed data by dividing the number of trials in which an event occurred by the total number of trials.
 - For instance, if it rained 3 out of 10 days, the probability of rain can be estimated as 30 percent.
 - Similarly, if 10 out of 50 email messages are spam, then the probability of spam can be estimated as 20 percent.
- The notation $P(A)$ is used to denote the probability of event A , as in $P(\text{spam}) = 0.20$.
- If the trial only has two outcomes that cannot occur simultaneously, such as heads or tails, or spam and ham (non-spam), then knowing the probability of either outcome reveals the probability of the other.
 - For example, given the value $P(\text{spam}) = 0.20$, we are able to calculate $P(\text{ham}) = 1 - 0.20 = 0.80$.
 - This works because the events spam and ham are **mutually exclusive** and **exhaustive**.

Probability

▶ Joint Probability

- ▶ Often, we are interested in monitoring several non-mutually exclusive events for the same trial. If some events occur with the event of interest, we may be able to use them to make predictions.
- ▶ Consider, for instance, a second event based on the outcome that the email message contains the word Viagra.
- ▶ For most people, this word is only likely to appear in a spam message; its presence in a message is therefore a very strong piece of evidence that the email is spam.

The Viagra circle does not completely fill the spam circle, nor is it completely contained by the spam circle. This implies that not all spam messages contain the word Viagra, and not every email with the word Viagra is spam.



Probability

▶ Joint Probability

- ▶ We know that 20 percent of all messages were spam and 5 percent of all messages contained Viagra.
 - Our job is to quantify the degree of overlap between these two proportions.
- ▶ In other words, we hope to estimate the probability of both $P(\text{spam})$ and $P(\text{Viagra})$ occurring, which can be written as $P(\text{spam} \cap \text{Viagra})$.
- ▶ Calculating $P(\text{spam} \cap \text{Viagra})$ depends on the **joint probability** of the two events, or how the probability of one event is related to the probability of the other. If the two events are totally unrelated, they are called **independent events**.
 - For instance, the outcome of a coin flip is independent from whether the weather is rainy or sunny.
- ▶ If all events were independent, it would be impossible to predict any event using the data obtained by another. On the other hand, **dependent events** are the basis of predictive modeling.
 - For instance, the presence of clouds is likely to be predictive of a rainy day, and the appearance of the word Viagra is predictive of a spam email.

Probability

▶ Joint probability – Independence

- ▶ If we assume that $P(\text{spam})$ and $P(\text{Viagra})$ were independent, we could then easily calculate $P(\text{spam} \cap \text{Viagra})$; the probability of both events happening at the same time.
 - (Note that they are not independent however!)
- ▶ Because 20 percent of all messages are spam, and 5 percent of all emails contain the word Viagra, we could assume that 5 percent of 20 percent ($0.05 * 0.20 = 0.01$), or 1 percent of all messages are spam containing the word Viagra.
- ▶ More generally, for independent events A and B , the probability of both happening is $P(A \cap B) = P(A) * P(B)$.
- ▶ In reality, it is far more likely that $P(\text{spam})$ and $P(\text{Viagra})$ are highly dependent, which means that this calculation is incorrect. So we need...

Probability

▶ Conditional Probability

- ▶ The relationships between dependent events can be described using **Bayes' Theorem**, as shown in the following formula.
- ▶ The notation $P(A|B)$ can be read as the probability of event A given that event B occurred.
- ▶ This is known as **conditional probability**, since the probability of A is dependent (that is, conditional) on what happened with event B.

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} = \frac{P(A \cap B)}{P(B)}$$

Probability

▶ Example

- ▶ Suppose that you were tasked with guessing the probability that an incoming email was spam.
- ▶ Without any additional evidence, the most reasonable guess would be the probability that any prior message was spam (that is, 20 percent in the preceding example).
- ▶ This estimate is known as the **prior probability**.
- ▶ Now, also suppose that you obtained an additional piece of evidence; you were told that the incoming message used the term Viagra.
- ▶ The probability that the word Viagra was used in previous spam messages is called the **likelihood** and the probability that Viagra appeared in any message at all is known as the **marginal likelihood**

Probability

▶ Example

- ▶ By applying Bayes' theorem to this evidence, we can compute a **posterior probability** that measures how likely the message is to be spam.
- ▶ If the posterior probability is greater than 50 percent, the message is more likely to be spam than ham, and it can potentially be filtered.
- ▶ The following formula is the Bayes' theorem for the given evidence:

$$P(\text{spam} \mid \text{Viagra}) = \frac{P(\text{Viagra} \mid \text{spam}) P(\text{spam})}{P(\text{Viagra})}$$

Diagram illustrating the components of Bayes' theorem for the given evidence:

- posterior probability** points to $P(\text{spam} \mid \text{Viagra})$
- likelihood** points to $P(\text{Viagra} \mid \text{spam})$
- prior probability** points to $P(\text{spam})$
- marginal likelihood** points to $P(\text{Viagra})$

Probability

▶ Example

- ▶ Of course we need information on how frequently has occurred as spam or ham.
- ▶ Lets assume these numbers:

	Viagra		
Frequency	Yes	No	Total
spam	4	16	20
ham	1	79	80
Total	5	95	100

	Viagra		
Likelihood	Yes	No	Total
spam	4 / 20	16 / 20	20
ham	1 / 80	79 / 80	80
Total	5 / 100	95 / 100	100

- ▶ The likelihood table reveals that $P(\text{Viagra}|\text{spam}) = 4/20 = 0.20$, indicating that the probability is 20 percent that a spam message contains the term Viagra.
- ▶ Additionally, since the theorem says that $P(B|A) * P(A) = P(A \cap B)$, we can calculate $P(\text{spam} \cap \text{Viagra})$ as $P(\text{Viagra}|\text{spam}) * P(\text{spam}) = (4/20) * (20/100) = 0.04$.
- ▶ This is four times greater than the previous estimate under the faulty independence assumption

Probability

▶ Example

- ▶ We aren't finished yet!
- ▶ To compute the posterior probability, $P(\text{spam}|\text{Viagra})$, we simply take $P(\text{Viagra}|\text{spam}) * P(\text{spam}) / P(\text{Viagra})$, or $(4/20) * (20/100) / (5/100) = 0.80$.
- ▶ Therefore, the probability is 80 percent that a message is spam, given that it contains the word Viagra.
- ▶ Therefore, any message containing this term should be filtered.
- ▶ *This is very much how commercial spam filters work, although they consider a much larger number of words simultaneously when computing the frequency and likelihood tables.*

Naïve Bayes

- ▶ Naïve Bayes
 - ▶ The naive Bayes algorithm is named as such because it makes a couple of "naive" assumptions about the data.
 - In particular, naive Bayes assumes that all of the features in the dataset are equally important and independent.
 - These assumptions are rarely true in most of the real-world applications.
 - ▶ However, in most cases when these assumptions are violated, naive Bayes still performs fairly well.
 - ▶ This is true even in extreme circumstances where strong dependencies are found among the features. Due to the algorithm's versatility and accuracy across many types of conditions, naive Bayes is often a strong first candidate for classification learning tasks.
 - ▶ Why? ...if a spam filter correctly identifies spam, does it matter that it was 51 percent or 99 percent confident in its prediction?

Naïve Bayes

▶ Example

- ▶ Let's extend our spam filter by adding a few additional terms to be monitored: money, groceries, and unsubscribe.
- ▶ The naive Bayes learner is trained by constructing a likelihood table for the appearance of these four words (W_1, W_2, W_3 , and W_4), as shown in the following diagram for 100 emails:

Likelihood	Viagra (W_1)		Money (W_2)		Groceries (W_3)		Unsubscribe (W_4)		Total
	Yes	No	Yes	No	Yes	No	Yes	No	
spam	4 / 20	16 / 20	10 / 20	10 / 20	0 / 20	20 / 20	12 / 20	8 / 20	20
ham	1 / 80	79 / 80	14 / 80	66 / 80	8 / 80	71 / 80	23 / 80	57 / 80	80
Total	5 / 100	95 / 100	24 / 100	76 / 100	8 / 100	91 / 100	35 / 100	65 / 100	100

- ▶ As new messages are received, the posterior probability must be calculated to determine whether they are more likely spam or ham, given the likelihood of the words found in the message text.
 - For example, suppose that a message contains the terms Viagra and Unsubscribe, but does not contain either Money or Groceries.

Naïve Bayes

▶ Example

- ▶ Using Bayes' theorem, we can define the problem as shown in the following formula, which captures the probability that a message is spam, given that Viagra = Yes, Money = No, Groceries = No, and Unsubscribe = Yes:

$$P(\text{Spam} | W_1 \cap \neg W_2 \cap \neg W_3 \cap W_4) = \frac{P(W_1 \cap \neg W_2 \cap \neg W_3 \cap W_4 | \text{spam}) P(\text{spam})}{P(W_1 \cap \neg W_2 \cap \neg W_3 \cap W_4)}$$

- ▶ For a number of reasons, this formula is computationally difficult to solve. As additional features are added, tremendous amounts of memory are needed to store probabilities for all of the possible intersecting events; imagine the complexity of a Venn diagram for the events for four words, let alone for hundreds or more...

Naïve Bayes

▶ Class-Conditional Independence

- ▶ The work becomes much easier if we can exploit the fact that naive Bayes assumes independence among events.
- ▶ Specifically, naive Bayes assumes **class-conditional independence**, which means that events are independent so long as they are conditioned on the same class value.
- ▶ Assuming conditional independence allows us to simplify the formula using the probability rule for independent events, which you may recall is $P(A \cap B) = P(A) * P(B)$. This results in a much easier-to-compute formulation, shown as follows:

$$P(\text{Spam} | W_1 \cap \neg W_2 \cap \neg W_3 \cap W_4) = \frac{P(W_1 | \text{spam}) P(\neg W_2 | \text{spam}) P(\neg W_3 | \text{spam}) P(W_4 | \text{spam}) P(\text{spam})}{P(W_1) P(\neg W_2) P(\neg W_3) P(W_4)}$$

$$P(\text{ham} | W_1 \cap \neg W_2 \cap \neg W_3 \cap W_4) = \frac{P(W_1 | \text{ham}) P(\neg W_2 | \text{ham}) P(\neg W_3 | \text{ham}) P(W_4 | \text{ham}) P(\text{ham})}{P(W_1) P(\neg W_2) P(\neg W_3) P(W_4)}$$

Naïve Bayes

▶ Example

- ▶ Using the values in the likelihood table, we can start filling numbers in these equations. Because the denominator is the same in both cases, it can be ignored for now. The overall likelihood of spam is then:
 - $(4/20) * (10/20) * (20/20) * (12/20) * (20/100) = 0.012$
- ▶ While the likelihood of ham given this pattern of words is:
 - $(1/80) * (66/80) * (71/80) * (23/80) * (80/100) = 0.002$
- ▶ Because $0.012 / 0.002 = 6$, we can say that this message is six times more likely to be spam than ham.
- ▶ However, to convert these numbers to probabilities, we need one last step. The probability of spam is equal to the likelihood that the message is spam divided by the likelihood that the message is either spam or ham:
 - $0.012 / (0.012 + 0.002) = 0.857$

Naïve Bayes

► Problems

- Suppose we received another message, this time containing the terms: Viagra, Groceries, Money, and Unsubscribe. The likelihood of spam is:
 - $(4/20) * (10/20) * (0/20) * (12/20) * (20/100) = 0$
- Surely this is a misclassification? Right?
- This problem might arise if an event never occurs for one or more levels of the class.
 - For instance, the term Groceries had never previously appeared in a spam message.
 - Consequently, $P(\text{spam}|\text{groceries}) = 0\%$.
- Because probabilities in naive Bayes are multiplied, this 0 percent value causes the posterior probability of spam to be zero, giving the word Groceries the ability to effectively nullify and overrule all of the other evidence.
- Even if the email was otherwise overwhelmingly expected to be spam, the absence of the word Groceries will always result in a probability of spam being zero.

Naïve Bayes

- ▶ Laplace Estimator
 - ▶ A solution to this problem involves using something called the **Laplace estimator**, which is named after the French mathematician *Pierre-Simon Laplace*.
 - ▶ The Laplace estimator essentially adds a small number to each of the counts in the frequency table, which ensures that each feature has a nonzero probability of occurring with each class.
 - ▶ Typically, the Laplace estimator is set to 1, which ensures that each class-feature combination is found in the data at least once.
 - The Laplace estimator can be set to any value, and does not necessarily even have to be the same for each of the features.

Naïve Bayes

▶ Laplace Estimator

- ▶ Using a Laplace value of 1, we add one to each numerator in the likelihood function. The total number of 1s must also be added to each denominator.
- ▶ The likelihood of spam is therefore:
 - $(5/24) * (11/24) * (1/24) * (13/24) * (20/100) = 0.0004$
- ▶ And the likelihood of ham is:
 - $(2/84) * (15/84) * (9/84) * (24/84) * (80/100) = 0.0001$
- ▶ This means that the probability of spam is 80 percent and the probability of ham is 20 percent; a more plausible result than the one obtained when Groceries alone determined the result.

Naïve Bayes

▶ Using Numeric Features

- ▶ Because naive Bayes uses frequency tables for learning the data, each feature must be categorical in order to create the combinations of class and feature values comprising the matrix.
- ▶ Since numeric features do not have categories of values, the preceding algorithm does not work directly with numeric data.
- ▶ One easy and effective solution is to **discretize** numeric features, which simply means that the numbers are put into categories known as **bins**.
 - For this reason, discretization is also sometimes called **binning**.
 - This method is ideal when there are large amounts of training data, a common condition when working with naive Bayes.

Examples

► Example in Python

```
# Using Numeric Features - Discretization
from sklearn.preprocessing import KBinsDiscretizer
from sklearn.naive_bayes import GaussianNB
from sklearn.datasets import load_iris
import seaborn as sns
import pandas as pd

# Load the iris dataset
iris = load_iris()
iris_data = pd.DataFrame(data=iris.data, columns=iris
    .feature_names)
iris_target = pd.Categorical.from_codes(iris.target, iris
    .target_names)

# Plot pairplot
sns.pairplot(pd.concat([iris_data, pd.DataFrame(iris_target,
    columns=['species'])], axis=1), hue='species')
plt.show()

# Discretize the features
kbins = KBinsDiscretizer(n_bins=3, encode='ordinal', strategy
    ='uniform')
iris_data_binned = kbins.fit_transform(iris_data)

# Train the Naive Bayes classifier
classifier = GaussianNB()
classifier.fit(iris_data_binned, iris_target)

# Predict using the classifier
predictions = classifier.predict(iris_data_binned)

# Display the confusion matrix
print(confusion_matrix(iris_target, predictions, labels=iris
    .target_names))
```

► Do you think this is good?

Examples

▶ Example – SMS spam

- ▶ As worldwide use of mobile phones has grown, a new avenue for electronic junk mail has been opened for disreputable marketers.
- ▶ These advertisers utilize **Short Message Service (SMS)** text messages to target potential consumers with unwanted advertising known as SMS spam.
- ▶ Developing a classification algorithm that could filter SMS spam would provide a useful tool for cellular phone providers.
- ▶ Since naive Bayes has been used successfully for email spam filtering, it seems likely that it could also be applied to SMS spam.
- ▶ However, relative to email spam, SMS spam poses additional challenges for automated filters.
- ▶ SMS messages are often limited to 160 characters, reducing the amount of text that can be used to identify whether a message is junk.
- ▶ The limit, combined with small mobile phone keyboards, has led many to adopt a form of SMS shorthand lingo, which further blurs the line between legitimate messages and spam.

Examples

▶ Data

- ▶ To develop the naive Bayes classifier, we will use data adapted from the SMS Spam Collection at:
 - <http://www.dt.fee.unicamp.br/~tiago/smsspamcollection/>
- ▶ This dataset includes the text of SMS messages along with a label indicating whether the message is unwanted.
- ▶ Junk messages are labeled spam, while legitimate messages are labeled ham. Some examples of spam and ham are shown in the following example:

Better. Made up for Friday and stuffed myself like a pig yesterday. Now I feel bleh. But at least its not writhing pain kind of bleh.

If he started searching he will get job in few days. He have great potential and talent.

I got another job! The one at the hospital doing data analysis or something, starts on monday! Not sure when my thesis will got finished

Examples

- ▶ Spam Messages

- ▶ The following is a sample spam messages:

Congratulations ur awarded 500 of CD vouchers or 125gift guaranteed & Freeentry 2 100 wkly draw txt MUSIC to 87066

December only! Had your mobile 1 1mths+? You are entitled to update to the latest colour camera mobile for Free! Call The Mobile Update Co FREE on 08002986906

Valentines Day Special! Win over £1 000 in our quiz and take your partner on the trip of a lifetime! Send GO to 83600 now. 150p/msg rcvd.

- ▶ Looking at the preceding sample messages, do you notice any distinguishing characteristics of spam? One notable characteristic is that two of the three spam messages use the word "free", yet the word does not appear in any of the ham messages.

Examples

▶ Load Data

```
import pandas as pd

# Load the dataset
sms_raw = pd.read_csv("C:/sms_spam.csv")

# Display the structure of the dataframe
print(sms_raw.info())

# Convert the type variable to a category
sms_raw["type"] = sms_raw["type"].astype("category")

# Display the structure of the 'type' column
print(sms_raw["type"].dtype)

# Display the count of each category
print(sms_raw["type"].value_counts())
```

- ▶ The type variable is currently a character vector. Since this is a categorical variable, it would be better to convert it to a factor, as shown in the following code:
- ▶ SMS messages are strings of text composed of words, spaces, numbers, and punctuation.
- ▶ Handling this type of complex data takes a large amount of thought and effort.

Examples

▶ Data Preparation

```
from sklearn.feature_extraction.text import CountVectorizer  
  
# Create a corpus  
vectorizer = CountVectorizer()  
sms_corpus = vectorizer.fit_transform(sms_raw['text'])
```

- ▶ One needs to consider how to remove numbers, punctuation, handle uninteresting words such as *and*, *but*, and *or*, and how to break apart sentences into individual words.
- ▶ This functionality has been provided by members of the Python community in text processing libraries like scikit-learn.
- ▶ The first step in processing text data involves creating a corpus, which refers to a collection of text documents. In our project, a text document refers to a single SMS message.



Examples

► Corpus

```
import re
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS

def text_preprocessing(text):
    # Convert to lowercase
    text = text.lower()
    # Remove numbers
    text = re.sub(r'\d+', '', text)
    # Remove punctuation
    text = re.sub(r'[^\w\s]', '', text)
    # Remove stopwords
    text = ' '.join([word for word in text.split() if word not in ENGLISH_STOP_WORDS])
    # Remove whitespace
    text = text.strip()
    return text

# Apply the text preprocessing
sms_raw['clean_text'] = sms_raw['text'].apply(text_preprocessing)

# Display the cleaned text
print(sms_raw[['text', 'clean_text']].head(3))
```

- If we print the corpus we just created, we will see that it contains documents for each of the SMS messages in the training data.
- First, we will convert all of the SMS messages to lowercase and remove any numbers.
- Stopwords are common words that are removed to help focus on the important words.

Examples

► Before / After

```
# Display the first 3 messages before cleaning  
print(sms_raw['text'].head(3))  
  
# Display the first 3 messages after cleaning  
print(sms_raw['clean_text'].head(3))
```

SMS messages before cleaning

```
> inspect(sms_corpus[1:3])  
[[1]]  
Hope you are having a good week.  
Just checking in  
[[2]]  
K..give back my thanks.  
[[3]]  
Am also doing in cbe only. But have  
to pay.
```

SMS messages after cleaning

```
> inspect(corpus_clean[1:3])  
[[1]]  
hope good week just checking  
[[2]]  
kgive back thanks  
[[3]]  
also cbe pay
```



Examples

▶ Training / Testing

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split

# Create the document-term matrix
vectorizer = CountVectorizer()
sms_dtm = vectorizer.fit_transform(sms_raw['clean_text'])

# Split the data into training and testing sets
sms_raw_train, sms_raw_test, y_train, y_test = train_test_split(
    sms_raw, sms_raw['type'], test_size=0.25, random_state=42)

# Display the shape of training and test sets
print(sms_raw_train.shape)
print(sms_raw_test.shape)
```

- ▶ This will tokenize the corpus and return the sparse matrix with the name `sms_dtm`.
- ▶ Since our data have been prepared for analysis, we now need to split the data into a training dataset and test dataset so that the spam classifier can be evaluated on data it had not seen previously.
- ▶ We'll divide the data into two portions: 75 percent for training and 25 percent for testing.
- ▶ We'll begin by splitting the raw data frame

mean

Examples

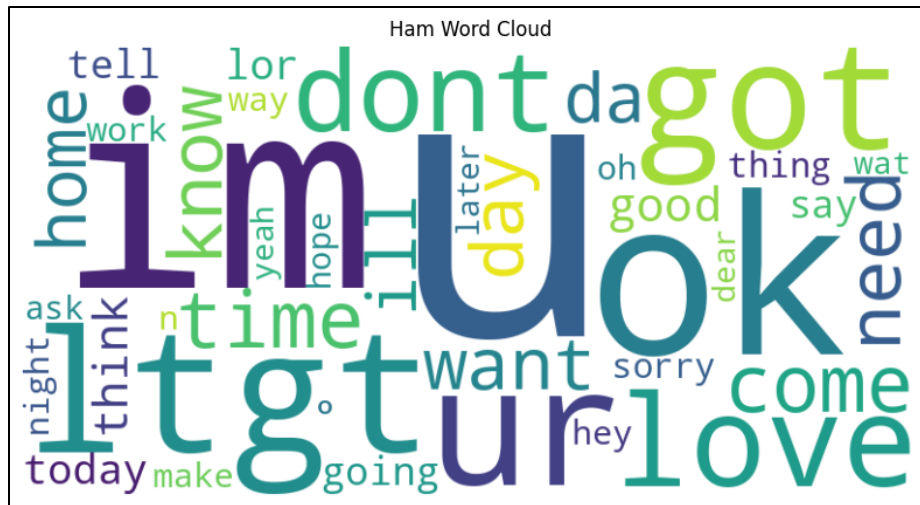
► Word Cloud

```
# Generate word clouds for spam and ham
spam = sms_raw_train[sms_raw_train['type'] == 'spam']
ham = sms_raw_train[sms_raw_train['type'] == 'ham']

spam_wordcloud = WordCloud(width=800, height=400, background_color='white', max_words=40, scale=3
    ).generate(' '.join(spam['clean_text']))
ham_wordcloud = WordCloud(width=800, height=400, background_color='white', max_words=40, scale=3
    ).generate(' '.join(ham['clean_text']))

# Display the word clouds
plt.figure(figsize=(10, 5))
plt.imshow(spam_wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Spam Word Cloud')
plt.show()

plt.figure(figsize=(10, 5))
plt.imshow(ham_wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Ham Word Cloud')
plt.show()
```



Examples

► Final Steps!

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Use TfidfVectorizer to filter terms that appear in at least 5 documents
vectorizer = TfidfVectorizer(min_df=5)
sms_dtm_train = vectorizer.fit_transform(sms_raw_train['clean_text'])
sms_dtm_test = vectorizer.transform(sms_raw_test['clean_text'])

# Display the shape of the document-term matrices
print(sms_dtm_train.shape)
print(sms_dtm_test.shape)
```

- The final step in the data preparation process is to transform the sparse matrix into a data structure that can be used to train a naive Bayes classifier. Currently, the sparse matrix includes over 7,000 features a feature for every word that appears in at least one SMS message.
- It's unlikely that all of these are useful for classification. To reduce the number of features, we will eliminate any words that appear in less than five SMS messages, or less than about 0.1 percent of records in the training data.

Examples

► Final Steps!

- The naive Bayes classifier is typically trained on data with categorical features.
- This poses a problem since the cells in the sparse matrix indicate a count of the times a word appears in a message.
- We should change this to a factor variable that simply indicates yes or no depending on whether the word appears at all. The following code defines a `convert_counts()` function to convert counts to factors:

```
# Convert counts to binary (0/1)
def convert_counts(x):
    return np.where(x > 0, 1, 0)

# Apply the function to convert counts to binary
sms_train = pd.DataFrame(sms_dtm_train.toarray(), columns
    =vectorizer.get_feature_names_out())
sms_train = sms_train.apply(convert_counts, axis=0)

sms_test = pd.DataFrame(sms_dtm_test.toarray(), columns
    =vectorizer.get_feature_names_out())
sms_test = sms_test.apply(convert_counts, axis=0)
```

Examples

Final Steps!

- ▶ Train the Naive Bayes classifier and predict the test set results.
- ▶ Use `confusion_matrix` and `classification_report` from `sklearn.metrics` to evaluate the model.

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import confusion_matrix, classification_report

# Train the Naive Bayes classifier
sms_classifier = MultinomialNB()
sms_classifier.fit(sms_train, y_train)

# Predict the test set results
sms_test_pred = sms_classifier.predict(sms_test)

# Display the confusion matrix and classification report
print(confusion_matrix(y_test, sms_test_pred))
print(classification_report(y_test, sms_test_pred))
```



Classification Report Output

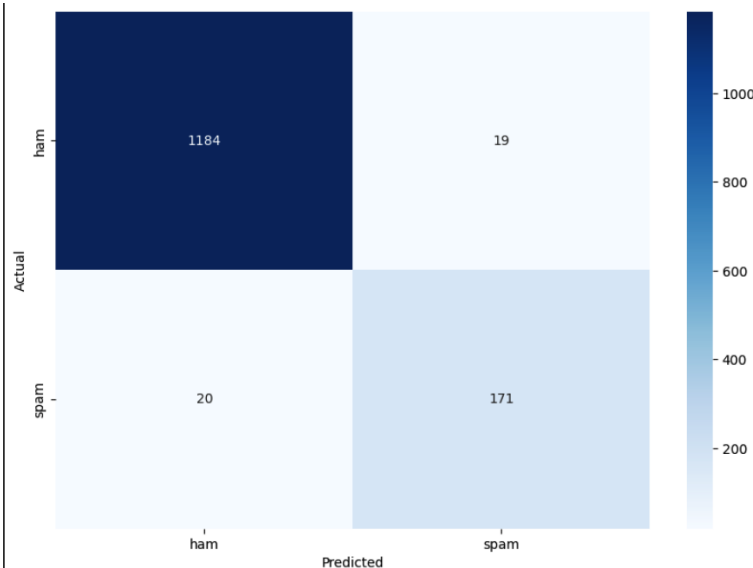
```
[[1184  19]
 [  20 171]]
```

	precision	recall	f1-score	support
ham	0.98	0.98	0.98	1203
spam	0.90	0.90	0.90	191
accuracy			0.97	1394
macro avg	0.94	0.94	0.94	1394
weighted avg	0.97	0.97	0.97	1394

Examples

► Results

- The confusion matrix provides a summary of the prediction results.
- Visualize the confusion matrix using `seaborn.heatmap`.



```
import seaborn as sns
import matplotlib.pyplot as plt

# Confusion matrix
conf_matrix = confusion_matrix(y_test, sms_test_pred)

# Plot the confusion matrix
plt.figure(figsize=(10, 7))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=['ham', 'spam'], yticklabels=['ham', 'spam'])
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```

Summary

Conclusions

- ▶ **Naive Bayes is:**
 - ▶ Really easy to implement and often works well.
 - ▶ Often a good first thing to try.
 - ▶ Commonly used as a “punching bag” for smarter algorithms.
- ▶ We looked at some text processing to convert data into a form that we can use with the Naive Bayes function in Python.
- ▶ The SMS case study exemplifies the reason why Naive Bayes is the standard for text classification: Directly out of the box, it performs surprisingly well.

Summary

- ▶ Introduction
- ▶ Probability
- ▶ Naive Bayes
- ▶ Examples

